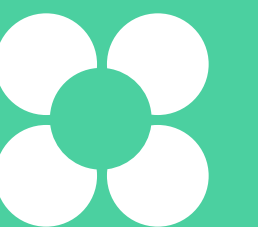
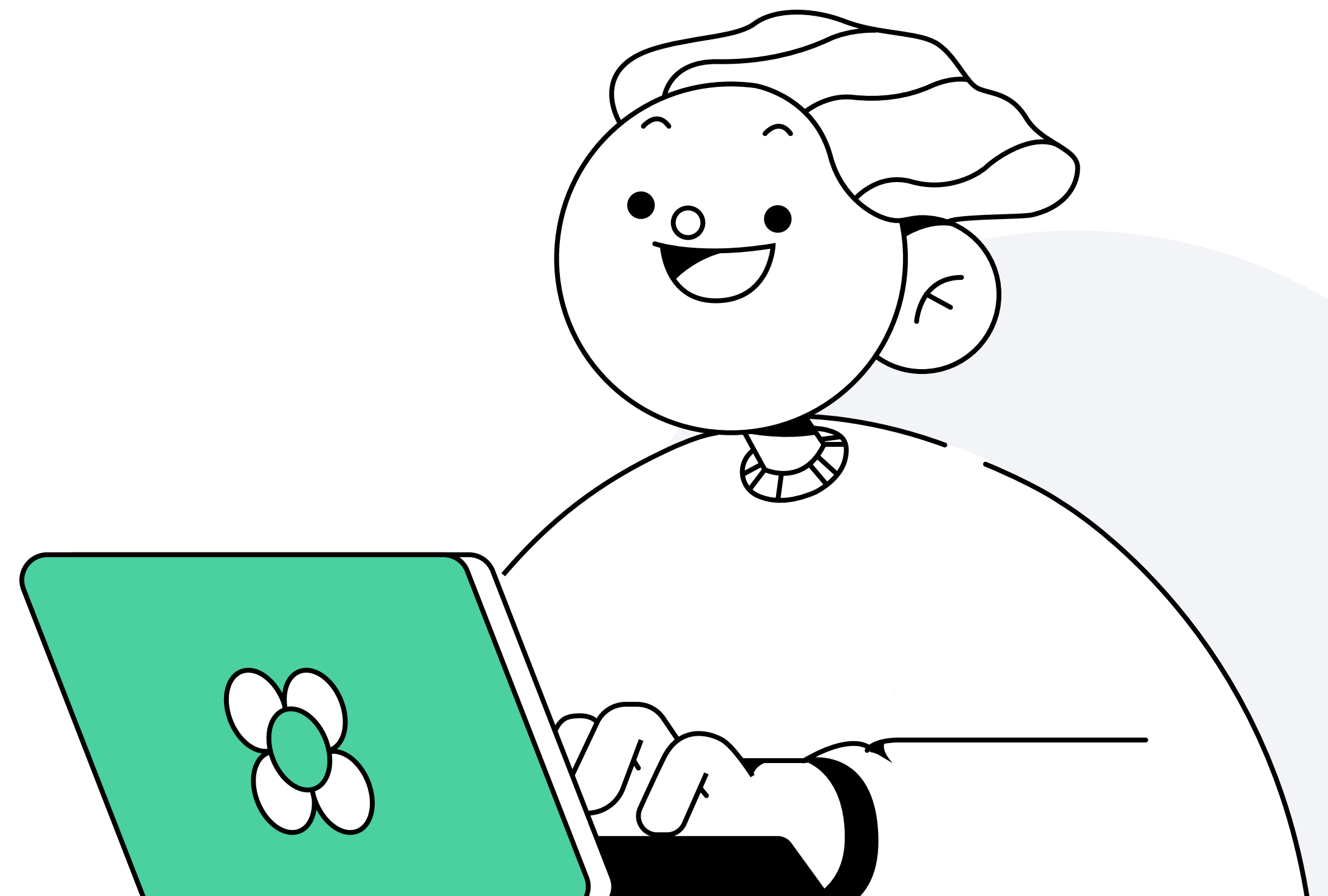


Условные операторы и циклы

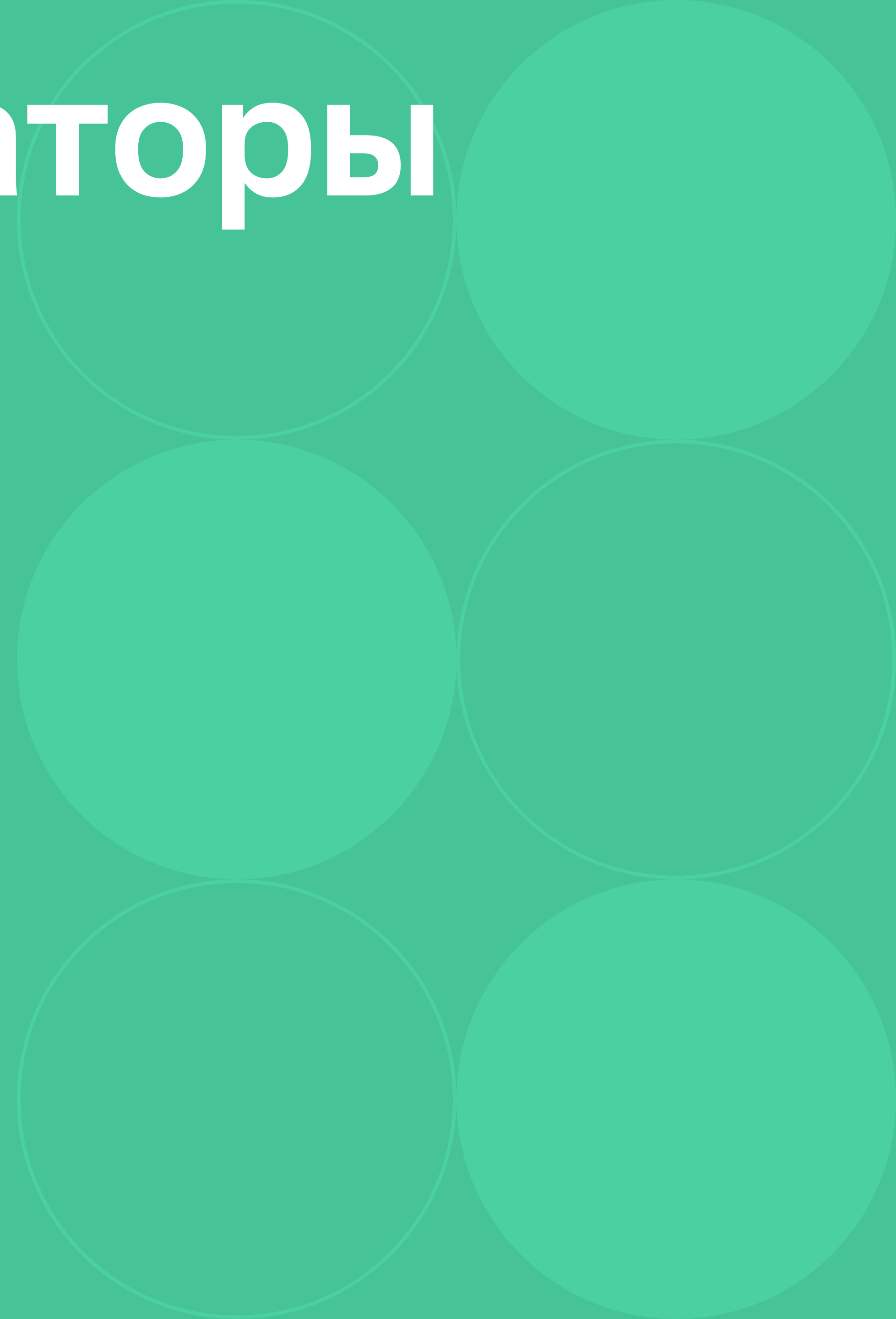


План занятия

- 1 Условные операторы
- 2 Циклы



Условные операторы



Условные операторы

Шаги выполнения программы:

- 1 Прочитать первое значение из консоли
- 2 Выполнить операции над ним
- 3 Прочитать второе значение из консоли
- 4 Выполнить операции над первым и вторым значениями
- 5 Вывести результат на экран

Эти шаги последовательны и исключают вариативность

Условные операторы

Если нам нужно выполнять разные действия после первого шага, в зависимости от прочитанного на первом шаге значения:

- ① Прочитайте первое значение из консоли
- ② Проверьте условия:
 - если на 1-м шаге введён ноль, завершите программу
 - если на 1-м шаге введён не ноль, продолжите выполнение
- ③ Прочитайте ещё одно значение
- ④ Выполните операции над первым и вторым значениями
- ⑤ Выведите результат на экран=

Условные операторы

Проблему ветвления или вариативности решают условные операторы:

1. `if`
2. `switch`
3. тернарный оператор

Оператор `if`

`if` — самый часто используемый условный оператор. Он проверяет условие, указанное в качестве аргумента `if` (условие).

Если результат вычисления условия верен, то программа продолжит выполнение кода, написанного внутри блока `{ код }`. Если не верен, то пропустит блок.

Пример: если температура чайника достигла 100 градусов, выполните код (выключите его):

```
if (temperatureTeapot == 100) {  
    // Код сработает, только если переменная temperatureTeapot = 100;  
}
```


Оператор **if**

Для проверки более чем одного условия операторы **if else** размещаются друг за другом.

Пример: если температура чайника достигла 100 градусов, выполните код (выключите его).
Если она равна 85 градусов, выведите на дисплей «Осталось 20 секунд».
Если она равна 60 градусов, включите дополнительный ТЭН.
Если же ни одно условие не подошло, то просто продолжайте кипятить воду в нём:

```
if (temperatureTeapot == 100) {  
    // Код в этом блоке сработает, только если переменная temperatureTeapot = 100  
}  
else if (temperatureTeapot == 85) {  
    // Код в этом блоке сработает, только если переменная temperatureTeapot = 85  
}  
else if (temperatureTeapot == 60) {  
    // Код в этом блоке сработает, только если переменная temperatureTeapot = 60  
}  
else {  
    // Код в этом блоке сработает, если ни одно условие выше не оказалось верным  
}
```

Оператор **if**

В некоторых случаях можно использовать короткую запись оператора **if** без **{ код }**, но только для однострочных инструкций.

Пример:

```
if (temperatureTeapot == 0) System.out.println("Температура чайника 0 градусов");
```

или более сложный пример:

```
if (temperatureTeapot == 100)
    System.out.println("Чайник вскипел");
else
    System.out.println("Температура чайника ещё меньше 100 градусов");
```

Оператор `switch`

`switch` — ещё один условный оператор, в качестве аргумента принимает переменную и сравнивает её значение в инструкциях оператора `case`

Дополнительно оператор `switch` имеет инструкцию `default`, определяющую вариант по умолчанию.

Аналогично `else` срабатывает в случае, если ни один из вариантов оператора `case` не совпал.

Визуально структура оператора `switch` отличается от оператора `if`, но эти операторы взаимозаменяемы, и в конечном счёте байт-код после компиляции будет одинаков

Оператор **switch**

Пример: в зависимости от температуры холодильника включите или отключите компрессор охлаждения в нём.

```
switch (refrigeratorTemperature) {  
    case 0:  
        // Код в этом блоке сработает, только если переменная  
        // refrigeratorTemperature = 0  
        break;  
    case -20:  
        // Код в этом блоке сработает, только если переменная  
        // refrigeratorTemperature = -20  
        break;  
    default:  
        // Код в этом блоке сработает, если ни одно условие выше не оказалось верным  
}
```

Оператор `switch`

В коде оператора `switch` мы объявили ещё незнакомый оператор `break`. Когда выполнение программы доходит до него, интерпретатор Java выходит из блока `case`.

Если не написать `break` после выполнения кода в блоке `case`, мы автоматически перейдём к блоку кода следующего `case`. При этом проверяться условие уже не будет.

Поэтому важно после каждого `case` не забывать добавлять оператор `break`

Тернарный оператор

Тернарный оператор — условный оператора без тела кода. Он используется, когда нужно присвоить значение переменной или вернуть значение из метода, не создавая избыточный код.

Конструкция состоит из условия (**условие**) и двух выражений. Первое выражение сработает, если условие верно, а второе — если ложно.

Выражения и условие разделены знаком вопроса ?

Тернарный оператор

Пример: если датчик температуры холодильника возвращает значение большее или равное нулю, подайте питание на компрессор охлаждения.

Если меньше нуля — не подавайте питание.

```
int switchOnRefrigiratorEngine = (refrigeratorTemperature >= 0) ? 1 : 0;
```

Если `(refrigeratorTemperature >= 0)` верно, то новое значение переменной `switchOnRefrigiratorEngine` будет равно `1`.

Если ложно, то новое значение переменной `switchOnRefrigiratorEngine` будет равно нулю.

То есть без явного указания `if` можно создать условие присвоения значения

Как правильно сравнивать переменные

В Java все типы данных делятся на две группы:

- примитивные типы
- ссылочные типы

Все примитивные типы сравниваются с помощью оператора `==`, а у всех ссылочных типов есть метод для их сравнения: метод `equals`.

Такая необходимость возникла вследствие того, что ссылочные типы более сложные, и их область хранения внутри JVM отличается от расположения примитивных типов.

К примитивным типам относятся почти все числовые типы, например `byte` и `int`

Как правильно сравнивать переменные

Пример: сравним температуру в микроволновой печи с температурой еды в ней.

```
int microwaveTemperature = 80;  
int foodTemperature = 10;  
if (microwaveTemperature == foodTemperature) {  
    System.out.println("Еда согрета");  
}
```

Строковый тип String является ссылочным, поэтому для его сравнения нужно использовать метод equals.

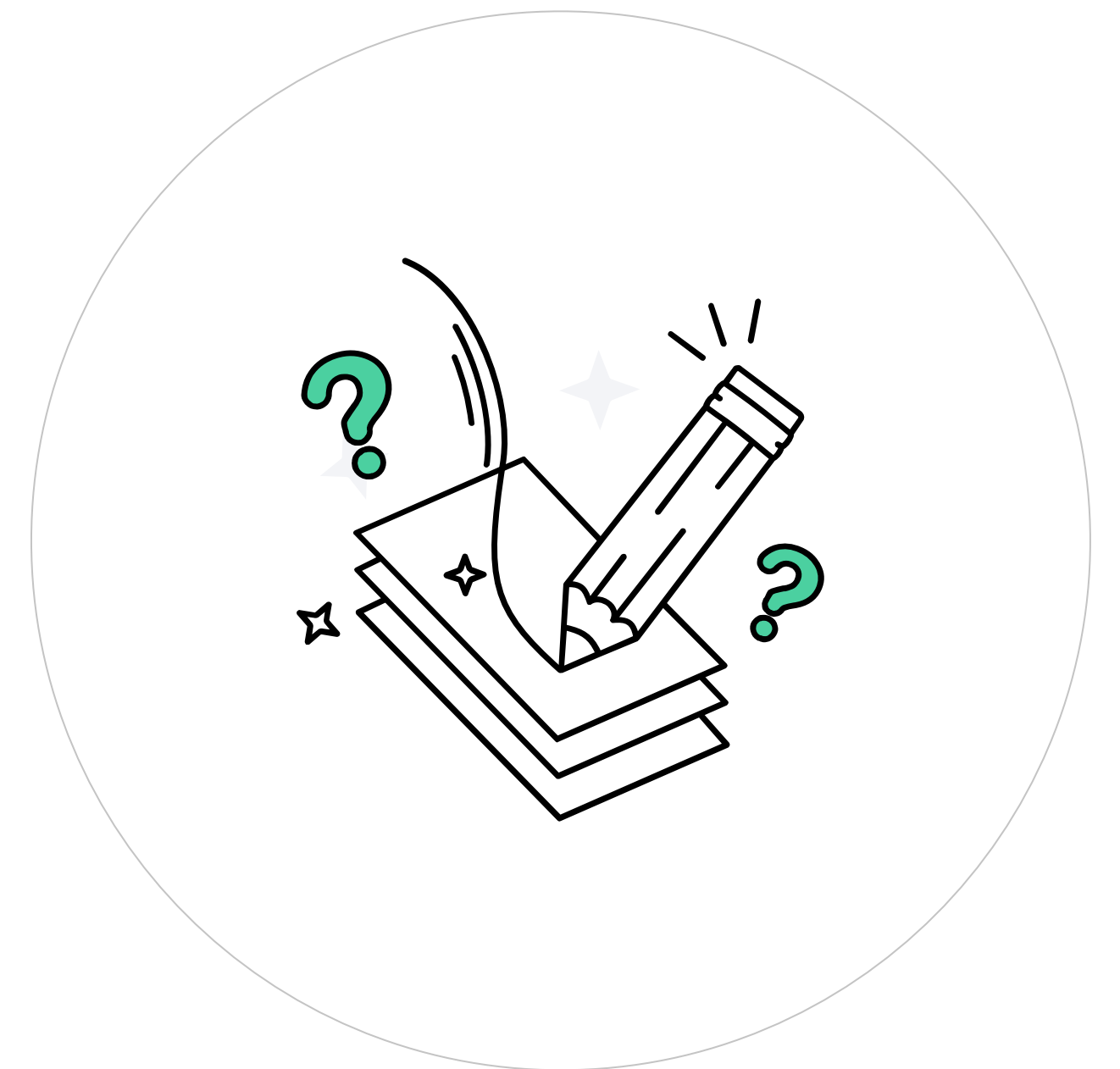
Пример: сравним метод разогрева еды, выбранный пользователем, с тем, что предоставлен на выбор:

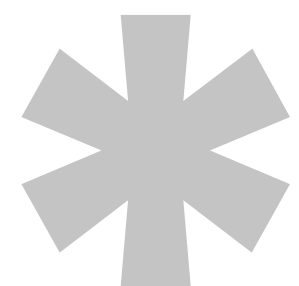
```
String userChoice = "экологично";  
if ("быстро".equals(userChoice)) {  
    System.out.println("Выбран быстрый режим");  
} else if ("экологично".equals(userChoice)) {  
    System.out.println("Выбран есо-режим");  
}
```

Логические операторы

Для формирования **сложных условий** в Java используются логические операторы:

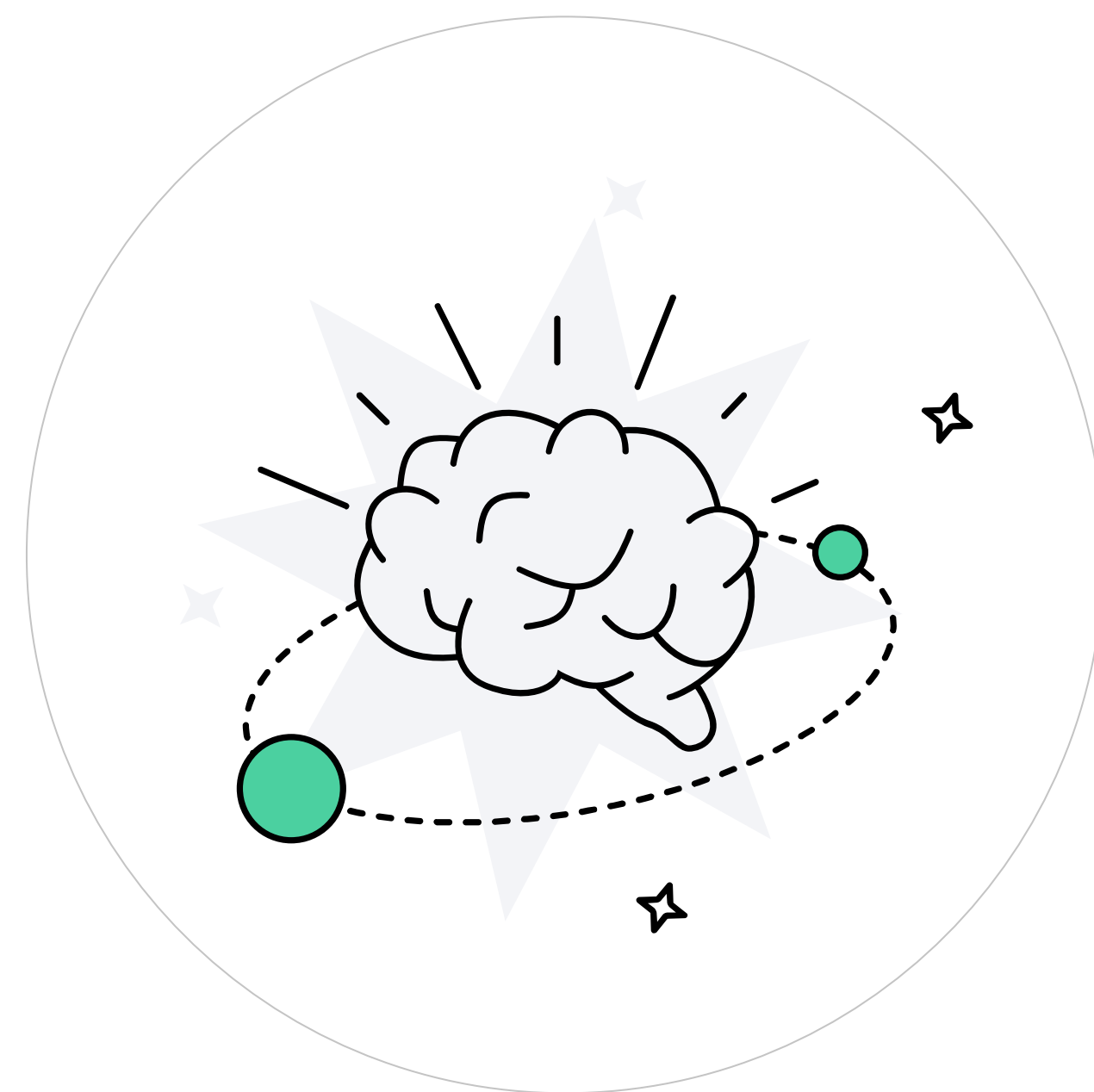
- Конъюнкция
- Дизъюнкция





**Конъюнкция - логическая операция,
часто называемая логическим
умножением или логическим И**

В Java логическое И представлено оператором &&



Пример конъюнкции

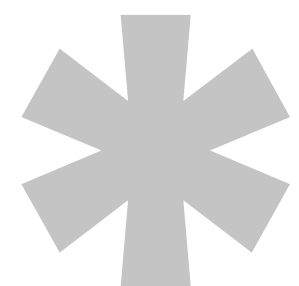
Проверка **X** на соответствие сразу двум значениям **X** $\geq$ **2** **И** **X** $\leq$ **5**
в Java будет представлена кодом:

```
int x = -10;  
  
if (x >= 2 && x <= 5) {  
    System.out.println("x соответствует условиям");  
} else {  
    System.out.println("x НЕ соответствует условиям");  
}
```

В результате получим:

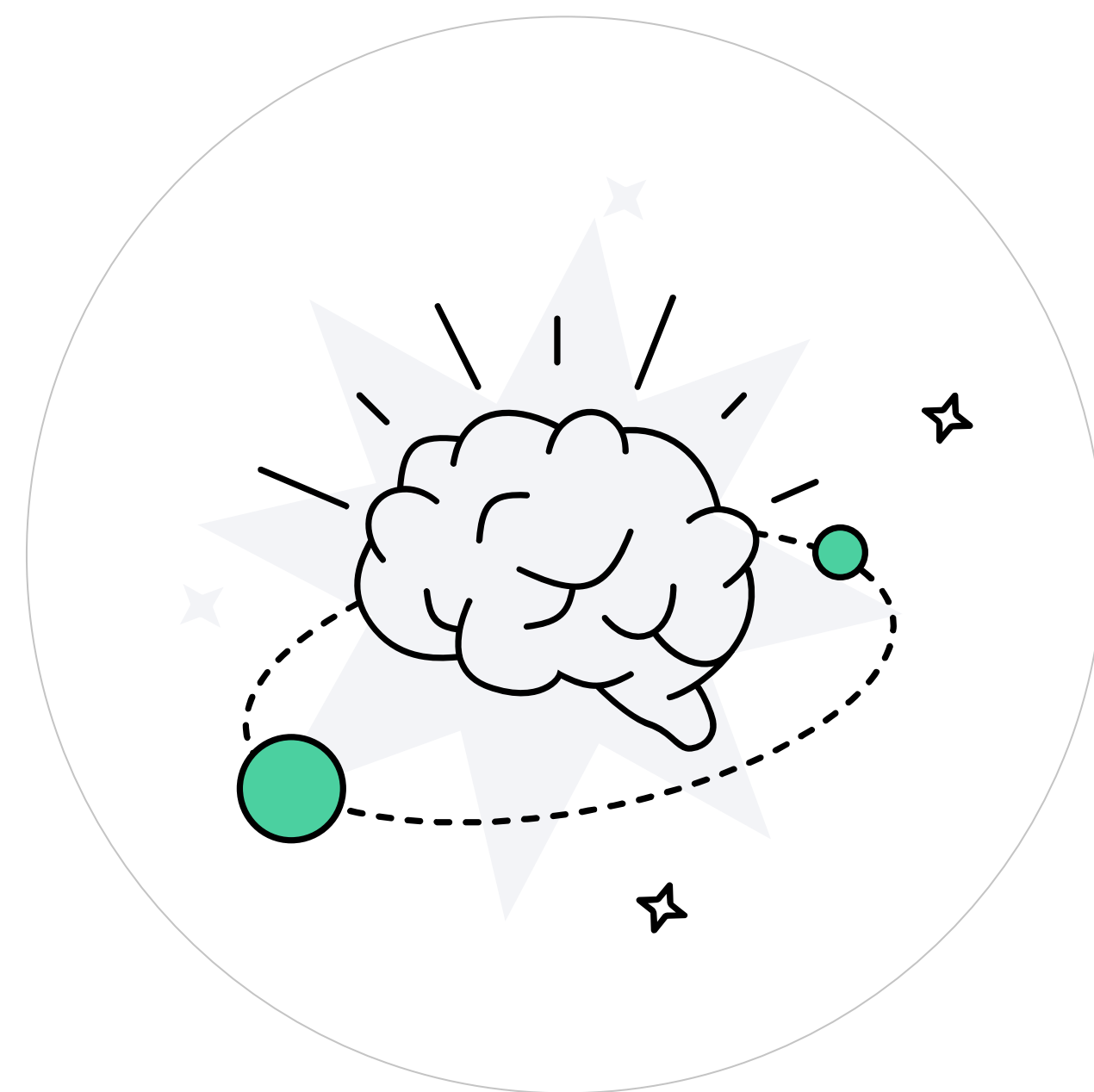
x НЕ соответствует условиям





**Дизъюнкция - логическая операция,
часто называемая логическим
сложением или логическим ИЛИ**

В Java логическое ИЛИ представлено оператором `||`



Пример дизъюнкции

Проверка **X** на соответствие хотя бы одному из двух значений **X >= 2** **или** **X <= 5** в Java будет представлена кодом:

```
int x = -10;  
  
if (x >= 2 || x <= 5) {  
    System.out.println("x соответствует условиям");  
} else {  
    System.out.println("x НЕ соответствует условиям");  
}
```

В результате получим:

x соответствует условиям



Таблицы истинности логических операторов

Отдельные части сложного условия могут принимать значение **true** или **false** в зависимости от результата проверки.

Все возможные результаты выполнения логических операций описаны в **таблицах истинности**, где **a** и **b** - части сложного условия, над которыми выполняются логические операции, а их значения **0** и **1** равны **true** и **false** соответственно.

a	b	a && b
0	0	0
0	1	0
1	0	0
1	1	1

a	b	a b
0	0	0
0	1	1
1	0	1
1	1	1

Циклы



Циклы

В Java, как и во многих других языках, для выполнения одного и того же кода много раз, помимо методов, есть циклы.

Цикл — это повтор выполняемого кода.

Циклы бывают нескольких видов:

- 1 while
- 2 do while
- 3 for
- 4 foreach

Цикл `while`, `do while`

Цикл `while` повторяет оператор или группу операторов, пока заданное условие является верным.

Цикл проверяет условие до выполнения тела цикла, поэтому цикл `while` называется циклом с предусловием.

Пример: выключите чайник, когда вода достигнет температуры в 100 градусов по цельсию:

```
int teapotTemperature = 27;
while (teapotTemperature < 100) {
    heatTeapot();
    teapotTemperature = getTeapotTemperature();
    System.out.println("Текущая температура чайника: " + teapotTemperature);
}
```

Пока температура чайника не достигнет 100 градусов, цикл будет выполняться

Цикл `while`, `do while`

С помощью цикла `do while` выполняется цикл `while`, за исключением того, что он проверяет условия в конце тела цикла. Поэтому цикл называется циклом с постусловием.

Пример: теперь мы хотим сварить макароны и будем их класть в кастрюлю до тех пор, пока она не заполнится доверху:

```
int pasta = 0; //Количество макарон в кастрюле
int potSize = 1000; //Ёмкость кастрюли
do {
    pasta = pasta + 100;
    System.out.println("Количество пасты в кастрюле: " + pasta);
} while (pasta < potSize);
```

Переменная `pasta` будет увеличиваться до тех пор, пока её значение не сравняется со значением переменной `potSize`

Цикл `do while` используется в случаях, если операторы в цикле нужно выполнить минимум 1 раз

Цикл **for**

Ещё одним оператором цикла является оператор **for**, он выполняет последовательность кода в нём столько раз, сколько определено в его переменной счётчика, и называется циклом со счётчиком. Такой цикл хорошо подходит, когда мы точно знаем, сколько раз мы хотим выполнить наш код.

Пример: если мы хотим порезать картошку на 1 000 маленьких кусочков, мы будем использовать цикл **for**:

```
for (int i = 0; i < 1000; i++) {  
    System.out.println("Количество кусочков картошки: " + i);  
}
```

i — переменная счётчика, **i++** — операция постинкремент, увеличение переменной на 1.

Плюс такого объявления цикла в том, что не нужно заранее в коде добавлять дополнительные переменные счётчика: как **i** из примера, они описываются и доступны только в пределах цикла **for**

Цикл `foreach`

Разновидность цикла `for` — цикл `foreach`. Он используется для перебора элементов массива или коллекции. Его сигнатура не включает присваивание начального значения и его последующего инкремента.

Пример: чтобы вывести слово «Борщ», перебрав все элементы массива, достаточно использовать цикл с перебором элементов этого массива

Разница между **for** и **foreach**

Напишем код с использованием цикла **for**:

```
char[] borsh = {'Б', 'о', 'р', 'щ'};  
for(int i = 0; i < borsh.length; i++) {  
    System.out.print(borsh[i]);  
}
```

Теперь напишем с использованием **foreach**:

```
char[] borsh = {'Б', 'о', 'р', 'щ'};  
for(char letter : borsh) {  
    System.out.print(letter);  
}
```

В нашем примере цикл **foreach** не требует дополнительных переменных, отвязанных от логики выражения, но при этом улучшает читаемость кода

Прерывания циклов

Иногда встречаются ситуации, когда нужно срочно прервать выполнение цикла или пропустить итерацию. Для этого были введены два оператора: `continue` и `break`.

Оператор `continue` пропускает выполнение итерации в цикле.

Пример: мы хотим приготовить суп и будем выбирать каждую вторую (чётную) морковь из ведра:

```
for (int i = 0; i < 10; i++) {  
    if (i % 2 != 0)  
        continue;  
    System.out.println("Морковь номер: " + i);  
}
```

При каждой проверке `if` мы проверяем, делится ли наше число без остатка на `2`. Если делится, значит `i` — чётное число

Прерывания циклов

Пример: при варке нашего супа мы добавляли разные овощи. При добавлении лука мы решили, что достаточно овощей:

```
String [] vegetables = {"курица", "петрушка", "лук", "сельдерей"}
for (String vegetable : vegetables) {
    if ("лук".equals(vegetable))
        break;
    System.out.println(vegetable);
}
System.out.println("Время варить!");
```

Этот цикл будет выполняться, пока значение **vegetable** не будет равно значению **"лук"**.

Как только выполнится условие оператора **if**, сработает вызов оператора **break**, программа выйдет из цикла и напечатает **"Время варить!"**

Общение с пользователем в цикле

Чтобы выводить и считывать данные в бесконечном цикле, нужно написать следующий код:

```
Scanner scanner = new Scanner(System.in);

while (true) {
    System.out.println("Введите значение"); //Выводим значение в консоль
    String value = scanner.nextLine(); //Считываем значение из консоли
    //в переменную TODO
}
```

Общение с пользователем в цикле

Чтобы выйти из такого цикла, нужно задать условие выхода, например ввод слова **end**:

```
Scanner scanner = new Scanner(System.in);

while (true) {
    System.out.println("Введите значение"); //Выводим значение в консоль
    String value = scanner.nextLine(); //Считываем значение из консоли
    //в переменную TODO
    if ("end".equals(value)) { //Если введённое значение равно `end`,
        //выходим из цикла
        break;
    }
}
```

Общение с пользователем в цикле

Чтобы выводить и считывать данные в цикле со счётчиком, напомним следующий код:

```
Scanner scanner = new Scanner(System.in);

for (int i = 0; i < 10; i++) {
    System.out.println("Введите значение"); //Выводим значение в консоль
    String value = scanner.nextLine(); //Считываем значение из консоли
    //в переменную
}
```

В этом случае нам не нужно условие выхода. Через 10 итераций цикл завершится, и программа выйдет из него